

图 7.1: 设备与驱动的绑定

项目中有三条总线：

- **platform**：SoC 内固定的设备（Mini UART、SDHOST、Arasan、DWC2），无法枚举，由板级代码静态描述，按名字匹配；
- **usb**：DWC2 枚举出的 USB 设备，按 VID/PID 或接口类别匹配；
- **sdio**：Arasan 枚举出的 SDIO function，按 vendor/device/function 匹配。

字符设备的主设备号见表 7.1。设备节点由内核的 devtmpfs 按已注册的驱动自动创建（`do_mounts.c`），`init` 只在 devtmpfs 不可用时才用 `mknod` 补建；`cat /proc/devices` 可以看到全部总线、设备、资源和已绑定的驱动。

表 7.1: 字符设备主设备号

major	名称	节点
1	console	`/dev/console`
2	tty（当前进程的控制终端）	`/dev/tty`
3	ttyS0（Mini UART）	`/dev/ttyS0`
4	input（evdev）	`/dev/input/event0`
5	camera	`/dev/video0`

7.2 上半部与下半部

中断处理程序运行时，本 CPU 的中断是关着的，而且它打断的可能是任何代码，所以不能睡眠、不能长时间运行。真正耗时的工作——解析 USB 报告、执行几十次 SDIO 命令、处理一个 TCP 段——要推迟到可以调度、可以睡眠的上下文中去做。项目实现了与 Linux 同名的 workqueue（`kernel/workqueue.h`）：

```
struct work_struct {
  void (*func)(struct work_struct *work);
  struct work_struct *next;
  int pending, running, rerun;
  struct workqueue *wq;
};
int  queue_work(struct workqueue *wq, struct work_struct *work);
int  queue_delayed_work(struct workqueue *wq, struct delayed_work *dw, uint64 delay);
void cancel_work_sync(struct work_struct *work);
```

每个 workqueue 有自己的内核线程（`kworker`），在 ps 中可以看到。几条原则贯穿所有驱动：

- 上半部只做三件事：确认中断来源、屏蔽或清除该中断、`queue_work()`；
- 每个设备使用私有的 work 和 workqueue，慢设备（如 SDIO 的 CMD53）不会阻塞别的下半部；
- 同一个 work 在执行期间再次被排队，只记一个 `rerun` 标志，不会无限堆积；
- 移除设备时先 `cancel_work_sync()`，等正在运行的 work 返回后才释放设备内存。

定时器中断每 10 ms 推进一次 delayed-work 时钟，TCP 重传、Wi-Fi 连接超时等都用 delayed work 实现，而